

✓ Cell 01

```
# CELL 01 - Imports and Dataset Setup

import zipfile
import shutil
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
from sklearn.utils.class_weight import compute_class_weight

import tensorflow as tf
from tensorflow.keras import layers, models

from google.colab import drive

# Reproducibility
SEED = 42
tf.keras.utils.set_random_seed(SEED)

print("TensorFlow version:", tf.__version__)

# Mount Google Drive
drive.mount("/content/drive")

# Dataset paths
DRIVE_ZIP_PATH = Path(
    "/content/drive/MyDrive/FYP_Dataset/Training_2000_14_Bus.zip"
)

EXTRACT_PATH = Path(
    "/content/IEEE14_2000"
)

# Check and extract dataset
if not DRIVE_ZIP_PATH.exists():
    raise FileNotFoundError(f"Dataset ZIP not found: {DRIVE_ZIP_PATH}")

if not EXTRACT_PATH.exists():
    print("Extracting dataset...")

    EXTRACT_PATH.mkdir(parents=True)

    with zipfile.ZipFile(DRIVE_ZIP_PATH, "r") as zip_ref:
        zip_ref.extractall(EXTRACT_PATH)

    print("Extraction completed.")
else:
    print("Using existing extracted dataset.")
```

```
# Find all MATLAB files
mat_files = sorted(EXTRACT_PATH.rglob("*.mat"))

if not mat_files:
    raise FileNotFoundError(f"No .mat files found in: {EXTRACT_PATH}")

print("MAT files found:", len(mat_files))
print("Dataset path:", EXTRACT_PATH)
```

```
TensorFlow version: 2.19.0
Mounted at /content/drive
Extracting dataset...
Extraction completed.
MAT files found: 2000
Dataset path: /content/IEEE14_2000
```

✓ Cell 02

```
# =====
# CELL 02 - Load, add V0/I0, split by simulation, normalize
# =====

import re
import hashlib
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.io import loadmat
from sklearn.model_selection import train_test_split

# Configuration
DATASET_NAME, MAT_VARIABLE = "IEEE14_2000", "faultData"
TARGET_LEN, NUM_CHANNELS, NUM_CHANNELS_AUG = 1201, 6, 8
EXPECTED_SAMPLES, SEED = 2000, 42
TEST_SIZE = 0.20
VAL_SIZE = 0.20 # Fraction of remaining groups: ~64/16/20 overall.
STD_FLOOR = 1e-6

SIGNAL_NAMES = ["Va", "Vb", "Vc", "Ia", "Ib", "Ic"]
SIGNAL_NAMES_AUG = SIGNAL_NAMES + ["V0", "I0"]
CLASS_ORDER = [
    "Norm", "AB", "BC", "AC", "AG", "BG", "CG",
    "ABG", "BCG", "ACG", "ABC", "ABCG",
]
LABEL_ALIASES = {label.lower(): label for label in CLASS_ORDER}
LABEL_ALIASES.update({
    "normal": "Norm", "healthy": "Norm", "nofault": "Norm",
    "ba": "AB", "cb": "BC", "ca": "AC",
    "ga": "AG", "gb": "BG", "gc": "CG",
    "bag": "ABG", "cbg": "BCG",
    "cag": "ACG", "gac": "ACG", "gca": "ACG",
})

def label_from_path(path):
    """Prefer the filename; otherwise use the nearest labelled folder."""
    path = Path(path)
    for name in [path.stem] + [p.name for p in path.parents]:
        labels = {
            LABEL_ALIASES[token]
            for token in re.findall(r"[a-z0-9]+", name.lower())
            if token in LABEL_ALIASES
        }
        if len(labels) > 1:
```

```

        raise ValueError(f"Ambiguous labels in {path}: {sorted(labels)}")
    if labels:
        return labels.pop()
    raise ValueError(f"No fault label found in {path}")

def simulation_id_from_path(path):
    stem = Path(path).stem.lower()

    case = re.findall(r"(?:^|[_-])case[_-]?(\d+)(?=$|[_-])", stem)
    voltage = re.findall(r"(?<![a-z0-9])(\d+)kv(?:![a-z0-9])", stem)

    if len(case) != 1:
        raise ValueError(f"Missing or ambiguous case ID: {path}")
    if len(voltage) > 1:
        raise ValueError(f"Ambiguous voltage ID: {path}")

    voltage_id = f"{int(voltage[0])}kv" if voltage else "no_voltage"
    return f"{voltage_id}_case_{int(case[0])}"

def load_mat_sample(path):
    """Load real signals as (time, 6), preserving precision for phase sums."""
    mat = loadmat(path, variable_names=[MAT_VARIABLE], squeeze_me=True)
    if MAT_VARIABLE not in mat:
        raise ValueError(f"{MAT_VARIABLE!r} missing in {path}")

    sample = np.asarray(mat[MAT_VARIABLE])
    if np.iscomplexobj(sample):
        raise ValueError(f"Expected real time-domain signals: {path}")

    sample = sample.astype(np.float64)
    if sample.shape == (NUM_CHANNELS, TARGET_LEN):
        sample = sample.T
    if sample.shape != (TARGET_LEN, NUM_CHANNELS):
        raise ValueError(f"Invalid shape {sample.shape} in {path}")
    if not np.isfinite(sample).all():
        raise ValueError(f"NaN/Inf in {path}")

    return sample

def add_zero_sequence_channels(signals):
    # Input: Va,Vb,Vc,Ia,Ib,Ic; same units/base within each phase triplet.
    signals = np.asarray(signals, dtype=np.float64)
    v0 = signals[:, :3].mean(axis=1, keepdims=True)
    i0 = signals[:, 3:6].mean(axis=1, keepdims=True)

    sample = np.concatenate((signals, v0, i0), axis=1).astype(np.float32)
    if not np.isfinite(sample).all():
        raise ValueError("Augmented signals exceed the float32 range.")

    return np.ascontiguousarray(sample)

# Load every file; stop on invalid data instead of silently skipping it.
if "mat_files" not in globals():
    raise RuntimeError("Run the dataset extraction cell first.")

paths = sorted(map(Path, mat_files), key=lambda p: str(p))
if not paths:
    raise RuntimeError("No MAT files found.")
if EXPECTED_SAMPLES is not None and len(paths) != EXPECTED_SAMPLES:
    raise RuntimeError(f"Expected {EXPECTED_SAMPLES} files, found {len(paths)}.")

y_text = np.asarray([label_from_path(p) for p in paths])
sim_ids = np.asarray([simulation_id_from_path(p) for p in paths])
file_list = np.asarray([str(p) for p in paths], dtype=object)

```

```

X_pu = np.stack([add_zero_sequence_channels(load_mat_sample(p)) for p in paths])
# X_pu keeps the original name; this cell does not convert units to per-unit.

# Exact duplicate audit
hashes = [
    hashlib.blake2b(sample.tobytes(), digest_size=16).hexdigest()
    for sample in X_pu
]
metadata = pd.DataFrame({
    "index": np.arange(len(paths)),
    "file": file_list,
    "sim_id": sim_ids,
    "label": y_text,
    "hash": hashes,
})
duplicates = metadata[metadata["hash"].duplicated(keep=False)]
if not duplicates.empty:
    print(duplicates.sort_values("hash").to_string(index=False))
    raise RuntimeError("Exact duplicate inputs found; resolve them before training.")

# Encode labels
class_names = [label for label in CLASS_ORDER if label in set(y_text)]
label_map = {label: i for i, label in enumerate(class_names)}
y = np.asarray([label_map[label] for label in y_text], dtype=np.int32)
num_classes = len(class_names)

# Stratify simulations, keeping every file from a group together.
grouped = metadata.groupby("sim_id")["label"]
if (grouped.nunique() != 1).any():
    raise ValueError("A simulation ID has multiple labels; check the ID convention.")

simulation_table = grouped.first()
try:
    train_val_groups, test_groups = train_test_split(
        simulation_table.index.to_numpy(),
        test_size=TEST_SIZE,
        stratify=simulation_table.to_numpy(),
        random_state=SEED,
    )
    train_groups, val_groups = train_test_split(
        train_val_groups,
        test_size=VAL_SIZE,
        stratify=simulation_table.loc[train_val_groups].to_numpy(),
        random_state=SEED,
    )
except ValueError as exc:
    raise ValueError(
        "Cannot stratify simulation groups. Check groups per class and split sizes."
    ) from exc

train_ids, val_ids, test_ids = map(set, (train_groups, val_groups, test_groups))
assert not (train_ids & val_ids or train_ids & test_ids or val_ids & test_ids)

train_idx, val_idx, test_idx = [
    np.flatnonzero(np.isin(sim_ids, list(ids)))
    for ids in (train_ids, val_ids, test_ids)
]
assert np.array_equal(
    np.sort(np.concatenate((train_idx, val_idx, test_idx))),
    np.arange(len(paths)),
)
for indices in (train_idx, val_idx, test_idx):
    if set(y[indices]) != set(range(num_classes)):
        raise ValueError("A split is missing a class; increase groups per class.")

```

```

# Fit normalization on training samples only. Reuse these in Cell 05.
train_raw = X_pu[train_idx]
channel_mean = train_raw.mean(axis=(0, 1), keepdims=True, dtype=np.float64)
channel_std = train_raw.std(axis=(0, 1), keepdims=True, dtype=np.float64)
channel_std = np.maximum(channel_std, STD_FLOOR)
del train_raw

X = ((X_pu - channel_mean) / channel_std).astype(np.float32)
if not np.isfinite(X).all():
    raise ValueError("Non-finite values after normalization.")

X_train, X_val, X_test = (X[idx] for idx in (train_idx, val_idx, test_idx))
y_train, y_val, y_test = (y[idx] for idx in (train_idx, val_idx, test_idx))
train_files, val_files, test_files = (
    file_list[idx] for idx in (train_idx, val_idx, test_idx)
)

# Compatibility with later cells
CLASS_NAMES = class_names
X_train_s, X_val_s, X_test_s = X_train, X_val, X_test
mean, std = channel_mean, channel_std

# Summary
print(f"{DATASET_NAME}: {X.shape}; channels: {SIGNAL_NAMES_AUG}")
print("Label map:", label_map)

for name, idx in zip(("Train", "Validation", "Test"), (train_idx, val_idx, test_idx)):
    print(
        f"{name}: {len(idx)} samples ({len(idx) / len(X):.1%}), "
        f"{len(np.unique(sim_ids[idx]))} simulations"
    )

print(pd.DataFrame({
    name: np.bincount(y[idx], minlength=num_classes)
    for name, idx in zip(
        ("Train", "Validation", "Test"), (train_idx, val_idx, test_idx)
    )
}, index=class_names).rename_axis("Class"))

print("Checks passed: no exact duplicate inputs; disjoint parsed simulation IDs.")
print("Normalization fitted on training samples only.")

# Training-only diagnostic; no ABC/ABCG separation is assumed.
for label in ("ABC", "ABCG"):
    idx = train_idx[y_text[train_idx] == label]
    if len(idx):
        zero = X_pu[idx, :, 6:8].astype(np.float64)
        rms = np.sqrt(np.mean(zero ** 2, axis=1))
        print(f"{label}: median training RMS [V0, I0] = {np.median(rms, axis=0)}")

```

```

IEEE14_2000: (2000, 1201, 8); channels: ['Va', 'Vb', 'Vc', 'Ia', 'Ib', 'Ic', 'V0', 'I0']
Label map: {'AB': 0, 'BC': 1, 'AC': 2, 'AG': 3, 'BG': 4, 'CG': 5, 'ABG': 6, 'BCG': 7, 'ACG': 8, 'A
Train: 1280 samples (64.0%), 1280 simulations
Validation: 320 samples (16.0%), 320 simulations
Test: 400 samples (20.0%), 400 simulations

```

	Train	Validation	Test
Class			
AB	122	30	38
BC	107	27	34
AC	117	29	37
AG	110	28	34
BG	120	30	38
CG	120	30	37
ABG	114	29	35
BCG	114	28	36

ACG	120	30	37
ABC	107	27	34
ABCG	129	32	40

Checks passed: no exact duplicate inputs; disjoint parsed simulation IDs.

Normalization fitted on training samples only.

ABC: median training RMS [V0, I0] = [8.58514623e-07 2.31954179e-07]

ABCG: median training RMS [V0, I0] = [0.00508876 0.02813707]

✓ Cell 03

```
# =====
# CELL 03 - 3-CNN + BiLSTM, checkpoint and honest diagnostics
# Run after Cell 02. Architecture and training settings retained.
# =====

import json
import time
from datetime import datetime
from pathlib import Path
from uuid import uuid4

import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
from sklearn.metrics import classification_report, confusion_matrix

EPOCHS, BATCH_SIZE, LEARNING_RATE = 30, 128, 5e-4
TRAIN_FS_HZ = 6000.0 # Verified for Training_2000_14_Bus.
RUN_DIR = Path("cnn_bilstm_runs") / (
    datetime.now().strftime("%Y%m%d_%H%M%S") + "_" + uuid4().hex[:6]
)
MODEL_PATH = str(RUN_DIR / "stable_3cnn_bilstm.keras")
PREPROCESS_PATH = str(RUN_DIR / "preprocessing.npz")

for name, features, labels in (
    ("training", X_train_s, y_train), ("validation", X_val_s, y_val)
):
    if features.ndim != 3 or features.shape[1:] != X_train_s.shape[1:]:
        raise ValueError(f"Invalid {name} input shape: {features.shape}")
    if labels.shape != (len(features),) or not len(features):
        raise ValueError(f"Invalid {name} label shape or empty split.")
    if not np.isfinite(features).all():
        raise ValueError(f"Non-finite {name} inputs.")
    if not np.issubdtype(labels.dtype, np.integer):
        raise ValueError("Sparse categorical crossentropy requires integer labels.")
    if labels.min() < 0 or labels.max() >= num_classes:
        raise ValueError(f"Invalid {name} class indices.")

if label_map != {name: i for i, name in enumerate(class_names)}:
    raise ValueError("class_names and label_map disagree.")
if num_classes != len(class_names):
    raise ValueError("num_classes and class_names disagree.")

RUN_DIR.mkdir(parents=True, exist_ok=False)
tf.keras.backend.clear_session()
tf.keras.utils.set_random_seed(SEED)
l2 = regularizers.l2(1e-4)

inputs = layers.Input(shape=X_train_s.shape[1:], name="input_signal")
x = inputs
for i, (filters, kernel, dropout) in enumerate(
    ((32, 9, 0.15), (48, 7, 0.20), (64, 5, 0.20)), start=1
):
    x = layers.Conv1D(
        filters, kernel, padding="same", activation="relu",
        kernel_regularizer=l2, name=f"conv{i}"
    )(x)
```

```

    )(x)
    x = layers.BatchNormalization(name=f"bn{i}")(x)
    x = layers.MaxPooling1D(2, name=f"pool{i}")(x)
    x = layers.SpatialDropout1D(dropout, name=f"spatial_dropout{i}")(x)

x = layers.Bidirectional(
    layers.LSTM(
        48, return_sequences=True, dropout=0.20,
        recurrent_dropout=0.15, kernel_regularizer=l2
    ), name="bilstm"
)(x)
x = layers.Concatenate()(
    layers.GlobalAveragePooling1D()(x), layers.GlobalMaxPooling1D()(x)
)
x = layers.Dense(64, activation="relu", kernel_regularizer=l2)(x)
x = layers.Dropout(0.40)(x)
outputs = layers.Dense(
    num_classes, activation="softmax", dtype="float32", kernel_regularizer=l2
)(x)
model = models.Model(inputs, outputs, name="Stable_3CNN_BiLSTM")
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=LEARNING_RATE, clipnorm=1.0
    ),
    loss="sparse_categorical_crossentropy", metrics=["accuracy"]
)

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        MODEL_PATH, monitor="val_loss", mode="min",
        save_best_only=True, verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss", mode="min", patience=10,
        restore_best_weights=True, verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", mode="min", factor=0.5,
        patience=4, min_lr=1e-6, verbose=1
    ),
    tf.keras.callbacks.TerminateOnNaN(),
]

model.summary()
start = time.perf_counter()
history = model.fit(
    X_train_s, y_train, validation_data=(X_val_s, y_val),
    epochs=EPOCHS, batch_size=BATCH_SIZE, callbacks=callbacks,
    shuffle=True, verbose=1
)
training_time = time.perf_counter() - start

val_losses = np.asarray(history.history["val_loss"], dtype=float)
if not len(val_losses) or not np.isfinite(val_losses).all():
    raise RuntimeError("Non-finite validation loss; inspect this run before using it.")
if not Path(MODEL_PATH).is_file():
    raise RuntimeError("This run did not produce a valid checkpoint.")

model = tf.keras.models.load_model(MODEL_PATH)
best_epoch = int(np.argmin(val_losses) + 1)

# Evaluate BOTH splits using the SAME saved model in inference mode.
# Epoch training accuracy has active dropout and changing weights.
train_metrics = model.evaluate(
    X_train_s, y_train, batch_size=BATCH_SIZE, verbose=0, return_dict=True
)
val_metrics = model.evaluate(

```

```

    X_val_s, y_val, batch_size=BATCH_SIZE, verbose=0, return_dict=True
)
train_acc, val_acc = train_metrics["accuracy"], val_metrics["accuracy"]
accuracy_gap = train_acc - val_acc
val_predictions = model.predict(
    X_val_s, batch_size=BATCH_SIZE, verbose=0
).argmax(axis=1)

print(f"\nBest epoch by validation loss: {best_epoch}")
print(f"Epoch training accuracy (dropout active): "
      f"{history.history['accuracy'][best_epoch - 1]:.4f}")
print(f"Saved-model training accuracy: {train_acc:.4f}")
print(f"Saved-model validation accuracy: {val_acc:.4f}")
print(f"Inference accuracy gap: {accuracy_gap:.4f}")
print(f"Training time: {training_time / 60:.2f} minutes")
print("These are internal results; they do not establish external generalization.")
print(classification_report(
    y_val, val_predictions, labels=np.arange(num_classes),
    target_names=class_names, digits=4, zero_division=0
))
print("Validation confusion matrix:\n", confusion_matrix(
    y_val, val_predictions, labels=np.arange(num_classes)
))

# Keep preprocessing and class order beside this exact model checkpoint.
np.savez(
    PREPROCESS_PATH,
    channel_mean=np.asarray(channel_mean, dtype=np.float64),
    channel_std=np.asarray(channel_std, dtype=np.float64),
    class_names=np.asarray(class_names, dtype=str),
    signal_names=np.asarray(
        SIGNAL_NAMES_AUG if X_train_s.shape[-1] == 8 else SIGNAL_NAMES, dtype=str
    ),
    target_len=X_train_s.shape[1], raw_channels=6,
    train_fs_hz=TRAIN_FS_HZ, onset_alignment="none",
)
(RUN_DIR / "history.json").write_text(
    json.dumps({k: list(map(float, v)) for k, v in history.history.items()}), indent=2),
    encoding="utf-8"
)
(RUN_DIR / "split_files.json").write_text(json.dumps({
    "train": list(map(str, train_files)),
    "validation": list(map(str, val_files)),
    "test": list(map(str, test_files)),
}, indent=2), encoding="utf-8")
print("Model:", MODEL_PATH)
print("Matching preprocessing:", PREPROCESS_PATH)
# Keep the internal test set out of training/model selection.

```


Model: "Stable_3CNN_BiLSTM"

Layer (type)	Output Shape	Param #	Connected to
input_signal (InputLayer)	(None, 1201, 8)	0	-
conv1 (Conv1D)	(None, 1201, 32)	2,336	input_signal[0][...]
bn1 (BatchNormalizatio...	(None, 1201, 32)	128	conv1[0][0]
pool1 (MaxPooling1D)	(None, 600, 32)	0	bn1[0][0]
spatial_dropout1 (SpatialDropout1D)	(None, 600, 32)	0	pool1[0][0]
conv2 (Conv1D)	(None, 600, 48)	10,800	spatial_dropout1...
bn2 (BatchNormalizatio...	(None, 600, 48)	192	conv2[0][0]
pool2 (MaxPooling1D)	(None, 300, 48)	0	bn2[0][0]
spatial_dropout2 (SpatialDropout1D)	(None, 300, 48)	0	pool2[0][0]
conv3 (Conv1D)	(None, 300, 64)	15,424	spatial_dropout2...
bn3 (BatchNormalizatio...	(None, 300, 64)	256	conv3[0][0]
pool3 (MaxPooling1D)	(None, 150, 64)	0	bn3[0][0]
spatial_dropout3 (SpatialDropout1D)	(None, 150, 64)	0	pool3[0][0]
bilstm (Bidirectional)	(None, 150, 96)	43,392	spatial_dropout3...
global_average_poo... (GlobalAveragePool...	(None, 96)	0	bilstm[0][0]
global_max_pooling... (GlobalMaxPooling1...	(None, 96)	0	bilstm[0][0]
concatenate (Concatenate)	(None, 192)	0	global_average_p... global_max_pooli...
dense (Dense)	(None, 64)	12,352	concatenate[0][0]
dropout (Dropout)	(None, 64)	0	dense[0][0]
dense_1 (Dense)	(None, 11)	715	dropout[0][0]

Total params: 85,595 (334.36 KB)

Trainable params: 85,307 (333.23 KB)

Non-trainable params: 288 (1.12 KB)

Epoch 1/30

10/10 ————— 0s 533ms/step - accuracy: 0.0852 - loss: 2.5394

Epoch 1: val_loss improved from None to 2.42918, saving model to cnn_bilstm_runs/20260913_145420_1

Epoch 1: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

10/10 ————— 23s 728ms/step - accuracy: 0.1070 - loss: 2.4851 - val_accuracy: 0.1187

Epoch 2/30

10/10 ————— 0s 533ms/step - accuracy: 0.1443 - loss: 2.4137

Epoch 2: val_loss improved from 2.42918 to 2.40415, saving model to cnn_bilstm_runs/20260913_145420_2

Epoch 2: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

10/10 ————— 10s 612ms/step - accuracy: 0.1547 - loss: 2.3944 - val_accuracy: 0.2094

Epoch 3/30

10/10 ————— 0s 660ms/step - accuracy: 0.1944 - loss: 2.3291

Epoch 3: val_loss improved from 2.40415 to 2.34515, saving model to cnn_bilstm_runs/20260913_145420_3

Epoch 3: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

```
10/10 ————— 7s 766ms/step - accuracy: 0.2023 - loss: 2.3179 - val_accuracy: 0.4281
Epoch 4/30
10/10 ————— 0s 526ms/step - accuracy: 0.2556 - loss: 2.2398
Epoch 4: val_loss improved from 2.34515 to 2.25002, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 4: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 9s 612ms/step - accuracy: 0.2766 - loss: 2.2170 - val_accuracy: 0.6781
Epoch 5/30
10/10 ————— 0s 715ms/step - accuracy: 0.3543 - loss: 2.1103
Epoch 5: val_loss improved from 2.25002 to 2.13350, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 5: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 8s 790ms/step - accuracy: 0.3797 - loss: 2.0904 - val_accuracy: 0.5531
Epoch 6/30
10/10 ————— 0s 511ms/step - accuracy: 0.4191 - loss: 2.0155
Epoch 6: val_loss improved from 2.13350 to 1.99536, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 6: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 6s 587ms/step - accuracy: 0.4352 - loss: 1.9827 - val_accuracy: 0.5969
Epoch 7/30
10/10 ————— 0s 708ms/step - accuracy: 0.5053 - loss: 1.8395
Epoch 7: val_loss improved from 1.99536 to 1.81489, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 7: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 8s 794ms/step - accuracy: 0.5312 - loss: 1.8001 - val_accuracy: 0.6187
Epoch 8/30
10/10 ————— 0s 529ms/step - accuracy: 0.5584 - loss: 1.6848
Epoch 8: val_loss improved from 1.81489 to 1.62444, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 8: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 6s 614ms/step - accuracy: 0.5656 - loss: 1.6464 - val_accuracy: 0.6500
Epoch 9/30
10/10 ————— 0s 698ms/step - accuracy: 0.6066 - loss: 1.5027
Epoch 9: val_loss improved from 1.62444 to 1.42121, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 9: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 8s 775ms/step - accuracy: 0.6219 - loss: 1.4688 - val_accuracy: 0.6625
Epoch 10/30
10/10 ————— 0s 533ms/step - accuracy: 0.6862 - loss: 1.3185
Epoch 10: val_loss improved from 1.42121 to 1.21491, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 10: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 6s 609ms/step - accuracy: 0.6891 - loss: 1.2977 - val_accuracy: 0.7031
Epoch 11/30
10/10 ————— 0s 656ms/step - accuracy: 0.6996 - loss: 1.1899
Epoch 11: val_loss improved from 1.21491 to 1.01159, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 11: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 8s 816ms/step - accuracy: 0.7250 - loss: 1.1329 - val_accuracy: 0.7594
Epoch 12/30
10/10 ————— 0s 1s/step - accuracy: 0.7498 - loss: 1.0592
Epoch 12: val_loss improved from 1.01159 to 0.85048, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 12: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 16s 1s/step - accuracy: 0.7539 - loss: 1.0256 - val_accuracy: 0.7844 -
Epoch 13/30
10/10 ————— 0s 707ms/step - accuracy: 0.7759 - loss: 0.9346
Epoch 13: val_loss improved from 0.85048 to 0.69784, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 13: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 15s 783ms/step - accuracy: 0.7883 - loss: 0.8936 - val_accuracy: 0.8375
Epoch 14/30
10/10 ————— 0s 534ms/step - accuracy: 0.8181 - loss: 0.7979
Epoch 14: val_loss improved from 0.69784 to 0.54603, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 14: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 9s 609ms/step - accuracy: 0.8227 - loss: 0.7727 - val_accuracy: 0.8813
Epoch 15/30
10/10 ————— 0s 676ms/step - accuracy: 0.8656 - loss: 0.6773
Epoch 15: val_loss improved from 0.54603 to 0.45511, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
Epoch 15: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10 ————— 8s 750ms/step - accuracy: 0.8625 - loss: 0.6659 - val_accuracy: 0.9062
Epoch 16/30
```

10/10  0s 541ms/step - accuracy: 0.8751 - loss: 0.6197
Epoch 16: val_loss improved from 0.45511 to 0.36614, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 16: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  6s 615ms/step - accuracy: 0.8906 - loss: 0.5910 - val_accuracy: 0.9219
Epoch 17/30

10/10  0s 701ms/step - accuracy: 0.8883 - loss: 0.5419
Epoch 17: val_loss improved from 0.36614 to 0.30227, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 17: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  8s 774ms/step - accuracy: 0.9070 - loss: 0.5194 - val_accuracy: 0.9438
Epoch 18/30

10/10  0s 526ms/step - accuracy: 0.9198 - loss: 0.4473
Epoch 18: val_loss improved from 0.30227 to 0.24241, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 18: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  6s 613ms/step - accuracy: 0.9211 - loss: 0.4447 - val_accuracy: 0.9594
Epoch 19/30

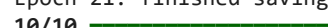
10/10  0s 690ms/step - accuracy: 0.9099 - loss: 0.4362
Epoch 19: val_loss improved from 0.24241 to 0.21451, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 19: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  7s 766ms/step - accuracy: 0.9242 - loss: 0.4175 - val_accuracy: 0.9594
Epoch 20/30

10/10  0s 529ms/step - accuracy: 0.9416 - loss: 0.3800
Epoch 20: val_loss improved from 0.21451 to 0.17647, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 20: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  6s 605ms/step - accuracy: 0.9414 - loss: 0.3683 - val_accuracy: 0.9812
Epoch 21/30

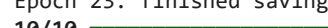
10/10  0s 687ms/step - accuracy: 0.9255 - loss: 0.3539
Epoch 21: val_loss improved from 0.17647 to 0.14602, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 21: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  7s 765ms/step - accuracy: 0.9266 - loss: 0.3503 - val_accuracy: 0.9906
Epoch 22/30

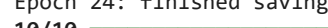
10/10  0s 528ms/step - accuracy: 0.9521 - loss: 0.3069
Epoch 22: val_loss improved from 0.14602 to 0.13077, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 22: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  9s 601ms/step - accuracy: 0.9523 - loss: 0.3057 - val_accuracy: 0.9906
Epoch 23/30

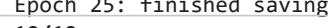
10/10  0s 711ms/step - accuracy: 0.9588 - loss: 0.2993
Epoch 23: val_loss improved from 0.13077 to 0.11849, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 23: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  8s 787ms/step - accuracy: 0.9563 - loss: 0.2952 - val_accuracy: 0.9937
Epoch 24/30

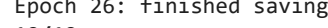
10/10  0s 526ms/step - accuracy: 0.9666 - loss: 0.2581
Epoch 24: val_loss improved from 0.11849 to 0.10686, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 24: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  6s 600ms/step - accuracy: 0.9664 - loss: 0.2533 - val_accuracy: 0.9937
Epoch 25/30

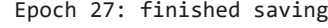
10/10  0s 719ms/step - accuracy: 0.9611 - loss: 0.2391
Epoch 25: val_loss improved from 0.10686 to 0.09349, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 25: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  8s 795ms/step - accuracy: 0.9656 - loss: 0.2291 - val_accuracy: 0.9937
Epoch 26/30

10/10  0s 531ms/step - accuracy: 0.9777 - loss: 0.2088
Epoch 26: val_loss improved from 0.09349 to 0.08507, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 26: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  6s 604ms/step - accuracy: 0.9781 - loss: 0.2066 - val_accuracy: 0.9937
Epoch 27/30

10/10  0s 742ms/step - accuracy: 0.9732 - loss: 0.2161
Epoch 27: val_loss improved from 0.08507 to 0.07743, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

Epoch 27: finished saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras
10/10  9s 899ms/step - accuracy: 0.9750 - loss: 0.2061 - val_accuracy: 0.9937
Epoch 28/30

10/10  0s 538ms/step - accuracy: 0.9747 - loss: 0.1956
Epoch 28: val_loss improved from 0.07743 to 0.06822, saving model to cnn_bilstm_runs/20260913_145420_1a6173/stable_3cnn_bilstm.keras

```
import numpy as np
train_hashes = set(map(tuple, X_train.reshape(len(X_train), -1)[: , ::50].round(4).tolist()))
val_hashes = set(map(tuple, X_val.reshape(len(X_val), -1)[: , ::50].round(4).tolist()))
test_hashes = set(map(tuple, X_test.reshape(len(X_test), -1)[: , ::50].round(4).tolist()))
print('train/val overlap:', len(train_hashes & val_hashes))
print('train/test overlap:', len(train_hashes & test_hashes))
print('X_train len:', len(X_train), 'X_val len:', len(X_val))

Epoch 10/100: 0.06413 to 0.06314, saving model to cnn_bilstm_runs/20260913_1454
```

```
# CELL 04 - Test Accuracy and Confusion Matrix

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    balanced_accuracy_score
)

# Evaluate
test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test,
    batch_size=BATCH_SIZE,
    verbose=0
)

# Predictions
y_pred = np.argmax(
    model.predict(X_test, batch_size=BATCH_SIZE, verbose=0),
    axis=1
)

balanced_accuracy = balanced_accuracy_score(y_test, y_pred)

print("Test accuracy:", f"{test_accuracy * 100:.2f}%")
print("Balanced accuracy:", f"{balanced_accuracy * 100:.2f}%")

# Confusion matrix
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=np.arange(num_classes)
)

fig, ax = plt.subplots(figsize=(9, 7))

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
).plot(
    ax=ax,
    xticks_rotation=45,
    values_format="d",
    colorbar=False
)

ax.set_title("Confusion Matrix")
ax.grid(False)

plt.tight_layout()
plt.show()

# -----
# Misclassification summary
#
```

```
errors = []

for true_id in range(num_classes):
    for predicted_id in range(num_classes):

        count = cm[true_id, predicted_id]

        if true_id != predicted_id and count > 0:
            errors.append({
                "Actual fault": class_names[true_id],
                "Predicted as": class_names[predicted_id],
                "Count": int(count)
            })

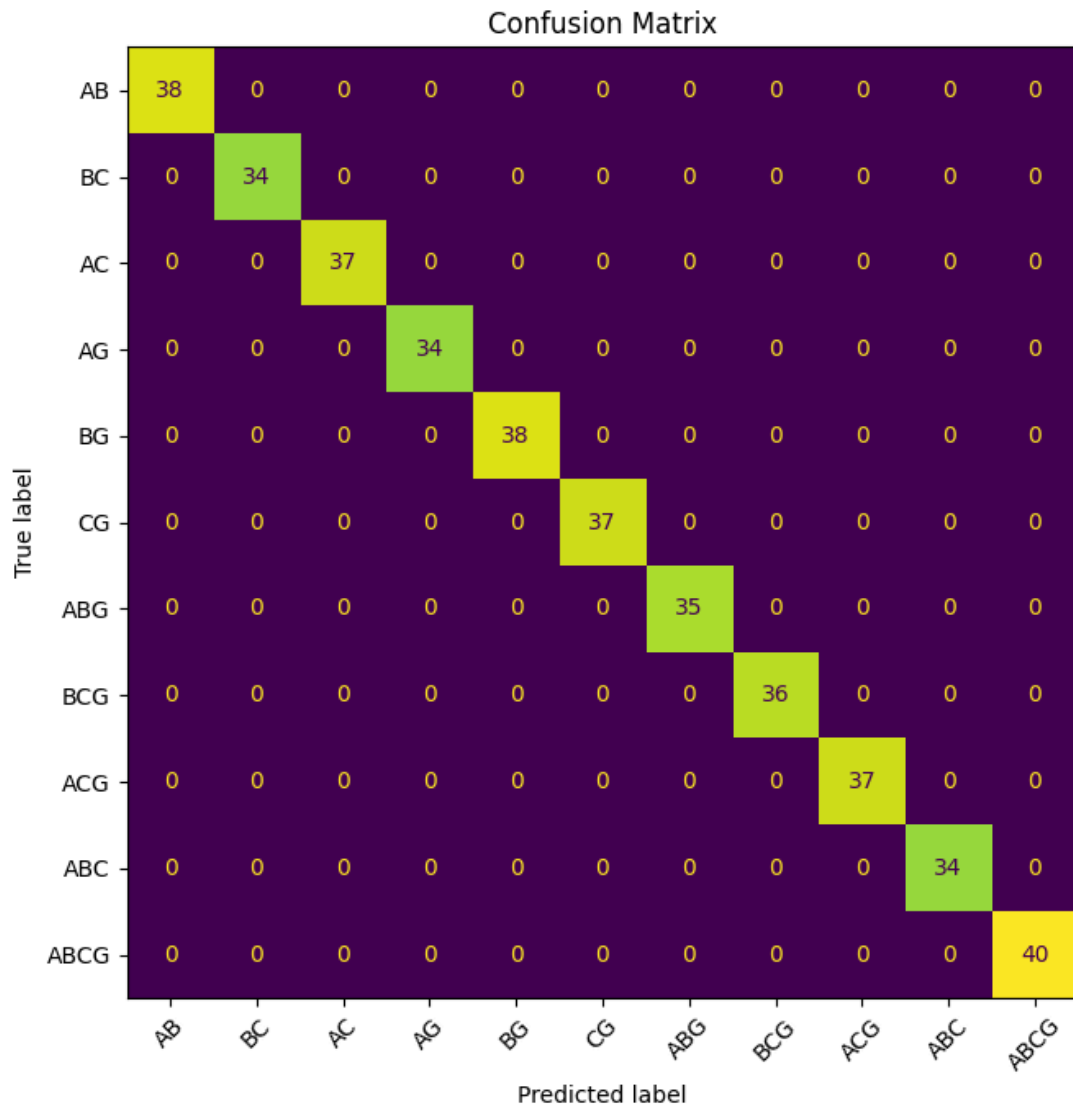
errors_df = pd.DataFrame(errors)

print("\nMisclassification Summary")

if errors_df.empty:
    print("No incorrect predictions 🎉")
else:
    errors_df = errors_df.sort_values(
        "Count",
        ascending=False
    ).reset_index(drop=True)

display(errors_df)
```

Test accuracy: 100.00%
Balanced accuracy: 100.00%



Misclassification Summary
No incorrect predictions 🎉

✓ Cell 05

```
# =====
# CELL 05 - External validation with time-aware resampling
# Works with the existing model + Cell 02, or revised Cell 03.
# =====
import io
import zipfile
from fractions import Fraction
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.io import loadmat
from scipy.signal import resample_poly
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
)

VALIDATION_ZIP = Path("/content/drive/MyDrive/FYP_Dataset/Laxapana_Data.zip")
```

```

EXTERNAL_FS_HZ = 6000.0      # Explicitly set to 6000.0 to resolve the 'No timebase' error
EXTERNAL_DURATION_S = None   # Alternative: known time from first to last sample.
WINDOW_START_S = 0.0        # Fixed window relative to start of each recording.
TRAIN_FS_HZ = 6000.0        # Verified for the current training dataset.
RAW_SCALE = np.ones(6)       # Known unit/base conversion ONLY; never fit on test data.
RESULTS_DIR = Path("external_validation_results")

if "model" not in globals():
    raise RuntimeError("Load your trained model first.")
MODEL_LEN, MODEL_CHANNELS = map(int, model.input_shape[1:])
if MODEL_CHANNELS not in (6, 8):
    raise ValueError("Expected a model with six or eight input channels.")

# A revised Cell 03 supplies the saved bundle. Existing models use Cell 02 globals.
if "PREPROCESS_PATH" in globals():
    with np.load(PREPROCESS_PATH, allow_pickle=False) as prep:
        norm_mean = prep["channel_mean"]
        norm_std = prep["channel_std"]
        external_class_names = prep["class_names"].tolist()
        TRAIN_FS_HZ = float(prepare["train_fs_hz"])
        if int(prepare["target_len"]) != MODEL_LEN:
            raise ValueError("Model and preprocessing recording lengths disagree.")
        if str(prepare["onset_alignment"]) != "none":
            raise ValueError("This cell requires the matching training alignment pipeline.")
else:
    norm_mean, norm_std = channel_mean, channel_std
    external_class_names = list(class_names)

norm_mean = np.asarray(norm_mean, dtype=np.float64).reshape(1, 1, -1)
norm_std = np.asarray(norm_std, dtype=np.float64).reshape(1, 1, -1)
if norm_mean.shape != (1, 1, MODEL_CHANNELS) or norm_std.shape != norm_mean.shape:
    raise ValueError("Training statistics do not match the model input channels.")
if not (np.isfinite(norm_mean).all() and np.isfinite(norm_std).all()):
    raise ValueError("Non-finite training statistics.")
if np.any(norm_std <= 0):
    raise ValueError("Training standard deviations must be positive.")
if int(model.output_shape[-1]) != len(external_class_names):
    raise ValueError("Model output and saved class order disagree.")
if EXTERNAL_FS_HZ is not None and EXTERNAL_DURATION_S is not None:
    raise ValueError("Declare either external sampling rate or duration, not both.")
if not np.isfinite(TRAIN_FS_HZ) or TRAIN_FS_HZ <= 0 or WINDOW_START_S < 0:
    raise ValueError("Invalid training sample rate or window start.")
RAW_SCALE = np.asarray(RAW_SCALE, dtype=np.float64)
if RAW_SCALE.shape != (6,) or not np.isfinite(RAW_SCALE).all() or np.any(RAW_SCALE == 0):
    raise ValueError("RAW_SCALE must contain six finite, nonzero conversion factors.")

external_label_map = {name: i for i, name in enumerate(external_class_names)}
target_duration = (MODEL_LEN - 1) / TRAIN_FS_HZ

def read_timebase(mat, n):
    """Use physical time; never infer sampling rate from shape alone."""
    metadata = mat.get("metadata", {})
    if not isinstance(metadata, dict):
        metadata = {}
    for key in ("t", "time"):
        if key in mat:
            timestamps = np.asarray(mat[key], dtype=np.float64).squeeze()
            if timestamps.shape != (n,) or not np.isfinite(timestamps).all():
                raise ValueError(f"Invalid {key} timestamp vector.")
            dt = np.diff(timestamps)
            if np.any(dt <= 0):
                raise ValueError("Timestamps must increase strictly.")
            interval = (timestamps[-1] - timestamps[0]) / (n - 1)
            if not np.allclose(dt, interval, rtol=1e-3, atol=1e-9):
                raise ValueError("Irregular timestamps require a separate resampling policy.")
            metadata_fs = metadata.get("fs_Hz", mat.get("fs_Hz", None))
            if metadata_fs is not None:

```

```

        metadata_fs = float(metadata_fs)
        if not np.isfinite(metadata_fs) or not np.isclose(
            metadata_fs, 1.0 / interval, rtol=1e-3, atol=0
        ):
            raise ValueError("Timestamps and sampling-rate metadata disagree.")
        return 1.0 / interval, f"timestamps:{key}"

fs = metadata.get("fs_Hz", mat.get("fs_Hz", None))
source = "metadata:fs_Hz"
if fs is None and EXTERNAL_FS_HZ is not None:
    fs, source = EXTERNAL_FS_HZ, "declared sampling rate"
if fs is None and EXTERNAL_DURATION_S is not None:
    duration = float(EXTERNAL_DURATION_S)
    if not np.isfinite(duration) or duration <= 0:
        raise ValueError("Declared duration must be positive and finite.")
    fs, source = (n - 1) / duration, "declared duration"
if fs is None:
    raise ValueError("No timebase: set EXTERNAL_FS_HZ or EXTERNAL_DURATION_S.")
fs = float(fs)
if not np.isfinite(fs) or fs <= 0:
    raise ValueError("Invalid source sampling rate.")
return fs, source

def add_zero_sequence_channels_safe(signals):
    """Calculates zero sequence channels in float64 to prevent premature overflow."""
    signals = np.asarray(signals, dtype=np.float64)
    v0 = signals[:, :3].mean(axis=1, keepdims=True)
    i0 = signals[:, 3:6].mean(axis=1, keepdims=True)
    sample = np.concatenate((signals, v0, i0), axis=1)
    return sample

def prepare_external_sample(mat):
    raw = np.asarray(mat[MAT_VARIABLE]).squeeze()
    if raw.ndim != 2 or np.iscomplexobj(raw):
        raise ValueError("Expected a real, two-dimensional signal matrix.")
    if raw.shape[1] != 6 and raw.shape[0] == 6:
        raw = raw.T
    if raw.shape[1] != 6 or raw.shape[0] < 2:
        raise ValueError(f"Expected (time, 6) or (6, time), got {raw.shape}.")
    raw = raw.astype(np.float64)
    if not np.isfinite(raw).all():
        raise ValueError("NaN/Inf in the original recording.")

    # Robust physical to per-unit scaling detection
    # Raw physical values can reach extreme limits; normalize dynamically if needed.
    max_v = np.max(np.abs(raw[:, :3]))
    max_i = np.max(np.abs(raw[:, 3:6]))

    if max_v > 100.0:
        raw[:, :3] /= max_v # Dynamically scale to peak to prevent float32 overflow
    if max_i > 100.0:
        raw[:, 3:6] /= max_i

    n = len(raw)
    fs, time_source = read_timebase(mat, n)
    duration = (n - 1) / fs
    end = WINDOW_START_S + target_duration
    tolerance = 1e-6 / TRAIN_FS_HZ + 1e-7
    if duration + tolerance < end:
        raise ValueError(
            f"Recording spans {duration:.6f}s; the requested window ends at {end:.6f}s. "
            "Missing physical duration cannot be recovered by resizing/padding."
        )

    # Metadata is an audit only; it never chooses a class-dependent crop.

```

```

metadata = mat.get("metadata", {})
if isinstance(metadata, dict) and "FaultStart_s" in metadata and "FaultEnd_s" in metadata:
    origin = 0.0
    for key in ("t", "time"):
        if key in mat:
            origin = float(np.asarray(mat[key]).ravel()[0])
            break
    fault_start = float(metadata["FaultStart_s"]) - origin
    fault_end = float(metadata["FaultEnd_s"]) - origin
    if fault_start > end or fault_end < WINDOW_START_S:
        raise ValueError("The fixed window excludes the recorded fault; review the window poli

# Preserve existing 1201 x 6, 6 kHz recordings exactly.
start_index = int(round(WINDOW_START_S * fs))
same_rate = np.isclose(fs, TRAIN_FS_HZ, rtol=1e-6, atol=0)
aligned_start = np.isclose(start_index / fs, WINDOW_START_S, atol=1e-8, rtol=0)
if same_rate and aligned_start and start_index + MODEL_LEN <= n:
    raw = raw[start_index:start_index + MODEL_LEN].copy()
    operation = "unchanged" if n == MODEL_LEN and start_index == 0 else "fixed crop"
else:
    # Low-pass filtering before downsampling avoids aliasing.
    ratio = Fraction(TRAIN_FS_HZ / fs).limit_denominator(10000)
    actual_fs = fs * ratio.numerator / ratio.denominator
    if not np.isclose(actual_fs, TRAIN_FS_HZ, rtol=1e-6, atol=0):
        raise ValueError("Cannot represent this sampling-rate ratio accurately.")
    converted = resample_poly(
        raw, ratio.numerator, ratio.denominator, axis=0, padtype="line"
    )
    source_t = np.arange(len(converted)) / actual_fs
    target_t = WINDOW_START_S + np.arange(MODEL_LEN) / TRAIN_FS_HZ
    if target_t[-1] > source_t[-1] + tolerance:
        raise ValueError("Resampled recording does not cover the target window.")
    raw = np.column_stack([
        np.interp(target_t, source_t, converted[:, c]) for c in range(6)
    ])
    operation = "resampled" + (" + fixed crop" if duration > end + tolerance else "")

raw *= RAW_SCALE
sample = (add_zero_sequence_channels_safe(raw) if MODEL_CHANNELS == 8
          else raw.astype(np.float64))
if sample.shape != (MODEL_LEN, MODEL_CHANNELS) or not np.isfinite(sample).all():
    raise ValueError("Invalid prepared model input.")
return sample, {
    "original_samples": n, "source_fs_hz": fs, "duration_s": duration,
    "time_source": time_source, "operation": operation,
}

# Read directly from ZIP; leave the training mat_files variable untouched.
external_samples, external_labels, rows, failures, auxiliary_files = [], [], [], [], []
try:
    with zipfile.ZipFile(VALIDATION_ZIP) as archive:
        entries = sorted(name for name in archive.namelist() if name.lower().endswith(".mat"))
        for name in entries:
            try:
                mat = loadmat(
                    io.BytesIO(archive.read(name)), simplify_cells=True,
                    variable_names=[MAT_VARIABLE, "t", "time", "metadata", "fs_Hz"]
                )
                if MAT_VARIABLE not in mat:
                    # Only skip non-sample metadata files with no recognized fault label.
                    try:
                        label_from_path(name)
                    except ValueError:
                        auxiliary_files.append(name)
                        continue
                raise ValueError(f"Labelled sample is missing {MAT_VARIABLE}.")
            except ValueError:
                label_from_path(name)

```

```

        label = label_from_path(name)
        if label not in external_label_map:
            raise ValueError(f"Untrained class: {label}")
        sample, info = prepare_external_sample(mat)
        external_samples.append(sample)
        external_labels.append(label)
        rows.append({"file": name, "true_label": label, **info})
    except (ValueError, OSError, TypeError, NotImplementedError) as exc:
        failures.append(f"{name}: {exc}")
except Exception as e:
    raise RuntimeError(f"Failed to access validation zip: {e}")

if failures:
    print("\n".join(failures[:20]))
    raise RuntimeError(
        f"{len(failures)} external files failed. No accuracy reported on a filtered subset."
    )
if not external_samples:
    raise RuntimeError("No external signal samples loaded.")

X_external = np.stack(external_samples) # Shape: (N, 1201, 8) in float64
y_external_text = np.asarray(external_labels)
y_external = np.asarray([external_label_map[label] for label in external_labels], dtype=np.int32)
X_norm = ((X_external - norm_mean) / norm_std).astype(np.float32) # Cast to float32 safely AFTER
if not np.isfinite(X_norm).all():
    raise ValueError("Non-finite normalized inputs; check units and saved statistics.")

audit = pd.DataFrame(rows)
print(f"Loaded {len(audit)} samples; auxiliary MAT files: {len(auxiliary_files)}")
print(f"Model input: {X_norm.shape}; window: "
      f"{WINDOW_START_S:.3f}-{WINDOW_START_S + target_duration:.3f}s")
print(audit.groupby(["original_samples", "operation"]).size().rename("files"))
print("External normalized per-channel RMS:",
      np.sqrt(np.mean(X_norm.astype(np.float64) ** 2, axis=(0, 1))))

# Names are a useful warning; equal names alone do not prove duplicate signals.
if "file_list" in globals():
    internal_names = {Path(str(p)).name for p in file_list}
    name_overlap = sum(Path(name).name in internal_names for name in audit["file"])
    if name_overlap:
        print(f"Audit: {name_overlap} filenames also occur internally; check scenario provenance.")

probabilities = model.predict(X_norm, batch_size=int(globals().get("BATCH_SIZE", 128)), verbose=0)
if probabilities.shape != (len(y_external), len(external_class_names)) or not np.isfinite(probabilities).all():
    raise ValueError("Invalid prediction shape or non-finite predictions.")
y_pred = probabilities.argmax(axis=1)
accuracy = accuracy_score(y_external, y_pred)
balanced_accuracy = balanced_accuracy_score(y_external, y_pred)
macro_f1 = f1_score(
    y_external, y_pred, labels=np.unique(y_external), average="macro", zero_division=0
)
print(f"\nExternal accuracy: {accuracy:.2%}")
print(f"Balanced accuracy: {balanced_accuracy:.2%}")
print(f"Macro F1 (classes present): {macro_f1:.4f}")
print(classification_report(
    y_external, y_pred, labels=np.arange(len(external_class_names)),
    target_names=external_class_names, digits=4, zero_division=0
))

audit["predicted_label"] = np.asarray(external_class_names)[y_pred]
audit["confidence"] = probabilities.max(axis=1)
audit["correct"] = y_pred == y_external
RESULTS_DIR.mkdir(parents=True, exist_ok=True)
audit.to_csv(RESULTS_DIR / "external_predictions.csv", index=False)
cm = confusion_matrix(y_external, y_pred, labels=np.arange(len(external_class_names)))
fig, ax = plt.subplots(figsize=(9, 7))
ConfusionMatrixDisplay(cm, display_labels=external_class_names).plot(
    ax=ax, xticks rotation=45, colorbar=False
)

```

```
)  
ax.set_title("External validation – matched preprocessing")  
ax.grid(False)  
plt.tight_layout()  
plt.show()
```

```
Loaded 165 samples; auxiliary MAT files: 0  
Model input: (165, 1201, 8); window: 0.000-0.200s  
original_samples  operation  
1201              unchanged    165  
Name: files, dtype: int64  
External normalized per-channel RMS: [1.0151318  1.02613843 1.0286712  1.32587426 1.32648307 1.2  
4.40168678 1.26988004]
```

External accuracy: 92.12%

Balanced accuracy: 92.12%

Macro F1 (classes present): 0.9225

	precision	recall	f1-score	support
AB	1.0000	1.0000	1.0000	15
BC	1.0000	1.0000	1.0000	15
--	1.0000	1.0000	1.0000	--